

Convolutional Neural Networks and the Transformer Architecture

From Spatial Feature Extraction to Sequence Transduction with Attention

Project report submitted by

Group 1

Aditya Arora (MA25M002)

Sagnik Barman (MA25M024)

Rumaiz Ibrahim K (MA25M023)

Adarsha Mandal (MA25M001)

under the guidance of

Dr. Priyanka Shukla

As a part of the course

Modeling Workshop

(MA5770)



Department of Mathematics
Indian Institute of Technology Madras
Chennai, 600036, India

April, 2026

Contents

1	Abstract	3
2	Introduction	4
3	Convolutional Neural Networks	4
3.1	Motivation: Why Not Plain Feedforward Networks?	4
3.2	The Three Fundamental Layer Types	5
3.3	Filters, Feature Maps, and Learned Kernels	6
3.4	Stride and Padding	7
3.5	Pooling Layers: Downsampling and Generalisation	7
3.6	Fully Connected Layers and the Classification Head	8
3.7	Complete CNN Architecture	8
3.8	ResNet: Deep Residual Networks	9
4	The Transformer Architecture	9
4.1	Motivation: Limitations of Recurrent Neural Networks	9
4.2	Overview of the Transformer	10
4.3	Tokenisation and Input Embeddings	11
4.4	Positional Encoding	12
4.5	Scaled Dot-Product Self-Attention	12
4.6	Multi-Head Attention	13
4.7	Masked Multi-Head Attention	13
4.8	Encoder-Decoder Architecture	13
4.9	Training vs. Inference	14
4.10	Experiment: English-to-Hindi Transformer from Scratch	14
5	Vision Transformer	15
6	Comparative Analysis: CNN vs. ViT on Brain MRI Classification	15
6.1	Problem Statement and Dataset	15
6.2	Model Performance	16
6.3	Counterfactual Attention Swap (CAS)	17
6.4	CNN-Guided CAS: Results	17
6.5	ViT-Guided CAS: Results	18
6.6	Discussion	18
7	Conclusion	18
8	References	19

1. Abstract

Abstract. This report provides a comprehensive treatment of two foundational architectures in modern deep learning: Convolutional Neural Networks (CNNs) and the Transformer model. Beginning with the limitations of standard feedforward networks for spatially structured data, we develop the theory of CNNs from first principles, examining convolution operations, filter learning, pooling, and the classification head. We then extend the discussion to Residual Networks (ResNet), which introduced shortcut connections to overcome the degradation problem in very deep networks. The second half of the report addresses sequential data and motivates the Transformer architecture by cataloguing the critical shortcomings of Recurrent Neural Networks. We analyse the self-attention mechanism, multi-head attention, positional encoding, and the full encoder-decoder stack, following the landmark paper by Vaswani et al. (2017). The Vision Transformer (ViT) is then introduced as the natural extension of the Transformer paradigm to image data. Finally, we present a comparative empirical study of CNN and ViT performance on a brain MRI classification task, analysed through a novel interpretability framework called Counterfactual Attention Swap (CAS).

2. Introduction

The past decade has witnessed two distinct but complementary revolutions in the design of neural architectures. The first was the widespread adoption of Convolutional Neural Networks, which demonstrated that the spatial structure of image data could be exploited through locally connected, weight-sharing filters to achieve dramatically superior performance on recognition tasks. The second revolution arrived with the introduction of the Transformer architecture by Vaswani et al. , which discarded recurrence and convolution entirely in favour of a mechanism called self-attention, enabling models to capture global dependencies across arbitrarily long sequences in a single computational step.

These two paradigms, initially developed for distinct modalities, CNNs for vision and Transformers for language, have since converged. The Vision Transformer (ViT) demonstrated that patch-based image representations, treated analogously to word tokens, can be processed by a standard Transformer encoder with state-of-the-art results on large-scale benchmarks. This convergence raises important empirical and interpretability questions: Do CNN-based and attention-based models rely on the same features when classifying complex medical images? If the salient region identified by one model is perturbed, how does the other model respond?

This report addresses both the theoretical foundations and the practical implications of these architectures. Section 3 develops the CNN framework in full, covering the roles of convolutional layers, pooling layers, padding, stride, and fully connected classification heads, before discussing the ResNet breakthrough. Section 4 presents the Transformer architecture step by step. Section 5 introduces the Vision Transformer. Section 6 presents a comparative analysis of CNN and ViT on brain MRI tumour classification using the CAS methodology.

3. Convolutional Neural Networks

3.1 Motivation: Why Not Plain Feedforward Networks?

Convolutional Neural Networks, also known as convnets, are a special class of neural network designed to process data with a known grid-like topology, such as one-dimensional time-series or two-dimensional images. Unlike fully connected networks, CNNs preserve and exploit the spatial relationships between input values by learning internal feature representations through small, local patches of the input . This spatial awareness is crucial for tasks where the position of a feature within the input domain carries information.

To appreciate why a dedicated architecture is necessary, consider a simple greyscale image of 32×32 pixels. A traditional feedforward network requires 1,024 input weights plus a bias merely to represent the first layer. More critically, however, the standard approach would flatten the two-dimensional pixel matrix into a long one-dimensional vector before processing. This operation irreversibly destroys all spatial structure: the network loses all knowledge of which pixels are neighbours, which edges they might form, and how objects are positioned relative to one another. Furthermore, border pixels receive fewer filter contributions than interior pixels, causing their features to be systematically under-represented, and a flattened network lacks the property of translation invariance.

CNNs address all of these deficiencies. By applying learnable filters in a sliding window fashion over the two-dimensional input, they naturally preserve spatial layout, share weights across positions (reducing the parameter count dramatically while enforcing translation invariance), and build a hierarchical representation in which early layers detect low-level features such as edges and textures, while deeper layers represent complex, semantically meaningful patterns.

3.2 The Three Fundamental Layer Types

A convolutional neural network is organised into three fundamental layer types, each serving a distinct role in the pipeline from raw pixels to final class probabilities.

The convolutional layer is the core building block. It applies a set of learnable filters, also called kernels, that slide across the input to produce feature maps. Each filter detects a particular local pattern - an edge at a specific orientation, a colour gradient, or a texture at every spatial location of the input.

Pooling layers follow sequences of one or more convolutional layers and serve to downsample the feature maps spatially. By summarising local neighbourhoods of activations, they reduce computational load, decrease the number of parameters in subsequent layers, and introduce a degree of spatial invariance to small translations and distortions.

Fully connected layers constitute the classification head at the tail of the network. Once convolutional and pooling blocks have produced a compact, high-level feature representation, the spatial dimensions are collapsed into a one-dimensional vector, which is then processed by one or more dense layers to produce the final output probability distribution over classes.

3.3 Filters, Feature Maps, and Learned Kernels

Each filter in a convolutional layer is a small weight matrix, typically 3×3 or 5×5 , whose values are learned during training. The filter is applied to the input by computing the elementwise dot product between the filter weights and a local patch of the input at each spatial position. The collection of these dot products across all positions forms a single feature map, also called an activation map, which encodes the presence and intensity of the pattern detected by that filter at each location.

When the convolutional layer is the first layer of the network, the patches contain raw pixel intensities. In deeper layers, the filter receives its input from the feature maps of the preceding layer, enabling the network to compose increasingly abstract representations. A filter in the second convolutional layer detects a particular combination of edge-responses from the first layer, corresponding to a corner, a curve, or a texture element.

Three classic kernel examples illustrate the principle. A sharpening kernel of the form $\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$ enhances fine detail by amplifying the difference between a pixel and its neighbours. The Laplacian edge-detection kernel $\begin{bmatrix} 1 & 1 & 1 \\ 1 & -8 & 1 \\ 1 & 1 & 1 \end{bmatrix}$ produces a strong response wherever pixel intensities change sharply. A uniform averaging kernel smooths the image by replacing each pixel with the mean of its neighbourhood. CNNs learn equivalent or superior filters autonomously through gradient descent.

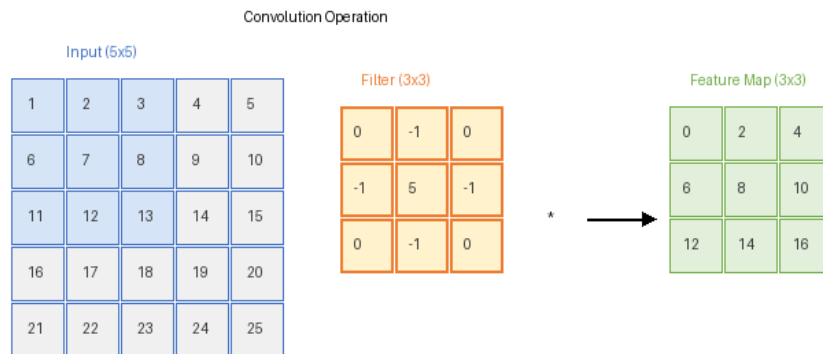


Figure 1: Illustration of the convolution operation: a 3×3 filter slides over the input to produce a feature map.

3.4 Stride and Padding

Two hyperparameters govern the spatial footprint of a convolutional layer's output relative to its input: stride and padding. Stride refers to the number of pixels by which the filter advances

at each step as it scans across the input. A stride of one moves the filter one pixel at a time; a stride of two halves the spatial dimensions. The output width under stride S , filter size F , padding P , and input width W_1 is given by the formula:

$$W_2 = \lfloor (W_1 - F + 2P) / S \rfloor + 1$$

Padding addresses a subtle but important boundary effect. Without any padding, pixels at the edges and corners of the input contribute to fewer filter positions than pixels in the interior, causing boundary information to be systematically under-represented. Valid padding ($P = 0$) applies no border and allows the feature map to shrink by $(F - 1)$ pixels per dimension at each layer. Same padding ($P = (F - 1)/2$) adds a border of zeros around the input so that the output has exactly the same spatial dimensions as the input when the stride is one. Padding is typically applied in every convolutional layer to ensure that spatial information is uniformly represented throughout the network depth.

3.5 Pooling Layers: Downsampling and Generalisation

Pooling layers downsample the previous layer's feature map. They always follow a sequence of one or more convolutional layers and are intended to consolidate the features learned and expressed in those maps. Pooling reduces the spatial resolution of the representation, which decreases the computational cost of subsequent layers and introduces a degree of invariance to small translations and deformations.

Three pooling variants are in common use. Max pooling takes the maximum value within each local patch, retaining the most prominent activation in that region. It is the most widely used form because the maximum is a strong indicator of whether a particular feature is present anywhere within the patch. Average pooling computes the arithmetic mean of all values in each patch, producing smoother feature maps that preserve more contextual information. Global average pooling reduces the entire spatial feature map for each channel to a single scalar value, eliminating all spatial dimensions at once; this is often used immediately before the final classification layer in modern architectures.

As an illustration, consider a 2×2 max pooling operation applied with stride 2 to the 4×4 input: $[[1,4,2,1],[5,8,3,4],[7,6,4,5],[1,3,1,2]]$. The pooling operation extracts the maximum from each non-overlapping 2×2 quadrant, yielding the 2×2 output $[[8,4],[7,5]]$.

3.6 Fully Connected Layers and the Classification Head

Fully connected layers are standard dense feedforward layers placed at the end of the CNN architecture. After the sequence of convolutional and pooling blocks has reduced the input volume to a compact, high-level feature representation, the three-dimensional feature map is flattened into a one-dimensional vector. This vector is then passed through one or more dense layers that learn nonlinear combinations of the extracted features.

The final fully connected layer typically produces one output unit per class, and its activations are passed through a softmax function to obtain a probability distribution over all classes. The softmax converts raw logits into non-negative values that sum to one, making them interpretable as class probabilities. The class with the highest probability is taken as the network's prediction.

3.7 Complete CNN Architecture

A complete CNN for image classification chains these components into a coherent pipeline. Consider a network designed for classifying 28×28 greyscale images into ten classes. The input passes first through a convolutional block consisting of a 3×3 convolutional layer with 32 filters and same padding, followed by a ReLU nonlinearity, producing 32 feature maps of size 28×28 . A 2×2 max pooling layer with stride 2 reduces each map to 14×14 . A second convolutional block with 64 filters of size 3×3 and same padding, again followed by ReLU and 2×2 max pooling, produces 64 feature maps of size 7×7 . The resulting volume of $64 \times 7 \times 7 = 3,136$ values is flattened and passed through a fully connected layer with 128 units and ReLU activation, before the final softmax layer with 10 output units.

The ReLU activation function, defined as $f(x) = \max(0, x)$, introduces the nonlinearity that allows the network to learn complex, non-linear decision boundaries. Without it, a deep stack of linear operations would collapse to a single linear transformation, regardless of depth.

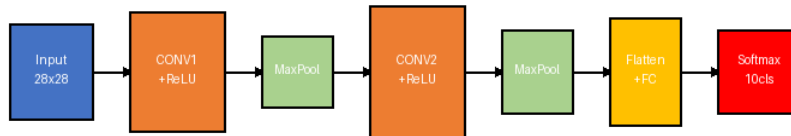


Figure 2: Complete CNN pipeline for image classification, from raw pixel input through convolutional and pooling blocks to the softmax output layer.

3.8 ResNet: Deep Residual Networks

A naive approach to improving CNN performance is to add more layers. In practice, however, beyond a certain depth, adding more layers to a plain CNN degrades training accuracy, not due to overfitting, but due to an optimisation difficulty termed the degradation problem. A 56-layer plain network performs worse on both training and test data than its 20-layer counterpart, because gradient signals shrink exponentially through backpropagation, making it difficult for early layers to learn even a simple identity mapping.

The ResNet solution reformulates the learning objective: rather than asking each stack of layers to learn an unrestricted mapping $H(x)$ directly, the layers learn the residual $F(x) = H(x) - x$. The output of each residual block is then:

$$H(x) = F(x) + x$$

implemented by a shortcut connection that adds the block's input directly to its output. When the residual branch learns $F(x) \approx 0$, the block reduces to a pure identity mapping, extra depth becomes free because layers can learn to do nothing when it is optimal to do so. An identity shortcut $y = F(x) + x$ is used when input and output dimensions match. A projection shortcut $y = F(x) + W_s x$ uses a 1×1 convolution W_s to match dimensions at downsampling boundaries.

The results were transformative. A 152-layer ResNet achieved a top-5 error rate of 3.57% on the ImageNet benchmark, winning first place in the ILSVRC 2015 challenges and surpassing estimated human-level performance of approximately 5.1% top-5 error.

4. The Transformer Architecture

4.1 Motivation: Limitations of Recurrent Neural Networks

While CNNs excel at tasks involving spatially structured data, a different class of architecture is required for sequential data such as natural language. Before the Transformer, the dominant approach was the Recurrent Neural Network (RNN), particularly its gated variants, the Long Short-Term Memory (LSTM) and the Gated Recurrent Unit (GRU). These models generate a sequence of hidden states by processing input tokens one at a time, each hidden state being a function of the current input and the previous hidden state. This sequential dependency means that computation cannot be parallelised across the time dimension.

Beyond computational inefficiency, RNNs suffer from two pathological gradient behaviours. Vanishing gradients arise when the gradient of the loss is the product of many values less than one, causing it to shrink exponentially toward zero as it propagates backward through time, making it difficult to capture long-range dependencies. Exploding gradients are the symmetric pathology: when the product of gradient values exceeds one at each step, parameter updates become erratically large and destabilise training. These fundamental limitations motivated the search for architectures that could process sequences in parallel and connect any two positions in a constant number of operations.

4.2 Overview of the Transformer

The Transformer, introduced in the seminal paper *Attention Is All You Need*, is the first sequence transduction model based entirely on attention mechanisms. There is no recurrence and no convolution, only attention. The architecture is fully parallelisable: all positions in the sequence are processed simultaneously, and any two positions are connected by a constant-length path through a single attention operation.

The model follows the standard encoder-decoder structure. The encoder reads the full input sequence and produces a rich, context-aware representation of each token. The decoder then generates the output sequence autoregressively. The base model uses six stacked encoder layers and six stacked decoder layers. On the WMT 2014 English-to-German translation benchmark, it achieved a BLEU score of 28.4, and 41.8 BLEU on English-to-French, while being trained in 3.5 days on eight P100 GPUs.



Figure 3: The Transformer encoder-decoder architecture. Left: encoder stack. Right: decoder stack with masked self-attention and cross-attention.

4.3 Tokenisation and Input Embeddings

Before any computation can begin, the input sentence must be converted into a numerical representation. The sentence is split into tokens and each token is mapped to a dense vector representation called an embedding. In the Transformer base model, each token maps to a vector of dimension $d_{\text{model}} = 512$. These embedding vectors are parameters of the model, initialised randomly and refined during training to capture semantic similarity. After training, semantically related words are mapped to nearby regions of the 512-dimensional embedding space.

An important implementation detail noted in [7] is that the embedding layer and the pre-softmax linear projection in the decoder share the same weight matrix, scaled by $\sqrt{d_{\text{model}}}$. This weight sharing reduces the total parameter count and improves training stability. The sentence 'the sky is blue' is tokenised into four indices, each mapped to a 512-dimensional embedding vector, yielding an input matrix of shape (4, 512).

4.4 Positional Encoding

Because the Transformer contains no recurrence and no convolution, it has no built-in mechanism for representing the order of tokens in a sequence. Without positional information, the model cannot distinguish 'the dog bites the man' from 'the man bites the dog'. Positional encoding is the mechanism by which token order is injected into the representation.

Vaswani et al. propose using fixed sinusoidal functions of different frequencies. The encoding for position pos and dimension index i is defined as:

$$PE(pos, 2i) = \sin(pos / 10000^{(2i / d_model)})$$
$$PE(pos, 2i+1) = \cos(pos / 10000^{(2i / d_model)})$$

The wavelengths form a geometric progression from 2π to $10000 \cdot 2\pi$, creating a unique fingerprint for every position. A key mathematical property is that, for any fixed offset k , $PE(pos + k)$ can be expressed as a linear function of $PE(pos)$, allowing the model to readily learn to attend by relative position. Sinusoidal encoding also allows the model to extrapolate to sequence lengths longer than those seen during training, something learned positional embeddings cannot do.

4.5 Scaled Dot-Product Self-Attention

Self-attention is the central mechanism of the Transformer. It enables each token to directly attend to every other token in the sequence, regardless of distance, and compute a context-aware representation as a weighted combination of all other tokens' values.

Formally, an attention function maps a query Q and a set of key-value pairs (K, V) to an output. The compatibility between a query and each key is measured by the dot product $Q \cdot K^T$, scaled by the inverse square root of the key dimension to prevent the softmax from saturating:

$$Attention(Q, K, V) = \text{softmax}(Q K^T / \sqrt{d_k}) \cdot V$$

The scaling by $\sqrt{d_k}$ is motivated by the observation that for large d_k , dot products grow large in magnitude, pushing the softmax into regions of very small gradient. Dividing by $\sqrt{d_k}$ counteracts this effect and keeps gradients well-behaved during training. In the encoder, $Q = K = V$ = the same input sequence (self-attention). In the cross-attention sub-layer of the decoder, Q comes from the decoder while K and V come from the encoder output, linking the two halves of the architecture.

4.6 Multi-Head Attention

A single attention function can only learn one type of relationship between tokens at a time. Multi-head attention addresses this limitation by running h independent attention functions in parallel, each operating on a different learned linear projection of the queries, keys, and values. The outputs of all heads are concatenated and projected back to d_{model} dimensions:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \cdot W^P$$
$$\text{where } \text{head}_i = \text{Attention}(Q W_i^Q, K W_i^K, V W_i^V)$$

In the base model, $h = 8$ heads are used, with $d_k = d_v = d_{\text{model}} / h = 512 / 8 = 64$ dimensions per head. Because each head operates on a lower-dimensional subspace, the total computational cost is similar to that of a single full-dimensional attention function. Different heads specialise empirically: some track syntactic relations, others capture semantic roles or co-reference, and still others attend to positional patterns.

4.7 Masked Multi-Head Attention

The decoder requires a form of attention that enforces causality: when generating the token at position i , the decoder must not attend to any position $j > i$. This causal constraint is enforced by masking the entries of the attention score matrix corresponding to future positions are set to $-\infty$ before the softmax is applied. After the softmax, these entries become zero, contributing nothing to the weighted sum of values. The resulting attention pattern is lower triangular, preserving the property that the decoder generates each token purely from its own past context.

4.8 Encoder-Decoder Architecture

The encoder is a stack of $N = 6$ identical layers, each containing two sub-layers: a multi-head self-attention sub-layer, and a position-wise fully connected feed-forward network (FFN). Each sub-layer is wrapped with a residual connection followed by layer normalisation. The feed-forward network applies two linear transformations with a ReLU activation in between:

$$\text{FFN}(x) = \max(0, x W_1 + b_1) W_2 + b_2$$

with $d_{\text{model}} = 512$ and inner dimension $d_{\text{ff}} = 2048$. The decoder is also a stack of $N = 6$ identical layers, but each layer contains three sub-layers: masked multi-head self-attention, cross-attention (to attend to the encoder's output), and a feed-forward network. Residual connections and layer normalisation are applied around each sub-layer. The final decoder output is passed through

a linear projection to vocabulary dimension and a softmax to produce a probability distribution over the next token.

4.9 Training vs. Inference

During training, the Transformer uses teacher forcing. The complete target sentence is provided to the decoder simultaneously, shifted one position to the right with a start-of-sequence token prepended. The causal mask ensures that position i sees only positions 0 to $i - 1$. A cross-entropy loss against the true target is computed for all positions simultaneously in a single forward pass. The Adam optimiser is used with a custom learning rate schedule that increases linearly for the first warmup steps then decreases proportionally to the inverse square root of the step number.

Inference proceeds autoregressively. Beginning with only the start-of-sequence token, the decoder generates one token at a time, appending each to its input and running again until an end-of-sequence token is generated or a maximum length is reached. Beam search is used to improve output quality: instead of greedily selecting the single highest-probability token, the top B candidate sequences are maintained and expanded simultaneously.

4.10 Experiment: English-to-Hindi Transformer from Scratch

To concretely illustrate the attention mechanism, a small Transformer was implemented and trained from scratch on a dataset of 60 English-Hindi sentence pairs. The architecture used $d_{\text{model}} = 128$, two encoder layers, two decoder layers, and four attention heads, yielding approximately 1,048,591 trainable parameters. The model was trained for 600 epochs using the Adam optimiser with learning rate 5×10^{-4} , $\beta_1 = 0.9$, $\beta_2 = 0.98$, and a batch size of 20 sentence pairs on a CUDA GPU.

The training loss dropped sharply from approximately 14.8 at epoch 1 to below 1.0 by epoch 50, indicating rapid acquisition of basic translation patterns. By epoch 300 the loss approached zero, confirming memorisation of the small training set, an expected outcome given the dataset size. Sample correct translations included: 'the sky is blue' $\rightarrow$ 'aasman neela hai'; 'i love you' $\rightarrow$ 'main tumse pyaar karta hoon'; 'the dog runs fast' $\rightarrow$ 'kutta tez daudta hai'.

Cross-attention heatmaps revealed clearly interpretable alignment patterns. Head 1 showed a strong link between the Hindi copula 'hai' and the English 'is' (weight 0.93). Head 2 aligned 'aasman' (sky) to 'sky' with weight 0.94, and 'neela' (blue) to 'blue' with weight 0.82. The average

attention matrix, pooled across all four heads, showed a diagonal score of only 0.26, reflecting the non-monotone word-order transformation required between English (SVO) and Hindi (SOV).

5. Vision Transformer

The Vision Transformer (ViT), introduced by Dosovitskiy et al. in An Image Is Worth 16×16 Words, adapts the Transformer architecture from natural language processing to image classification. The core insight is that an image can be partitioned into a sequence of fixed-size patches, each treated analogously to a word token, and the resulting sequence processed by a standard Transformer encoder without any convolution.

Concretely, an input image of spatial resolution $H \times W$ is divided into patches of size $P \times P$ (typically $P = 16$), yielding $N = HW/P^2$ patches. The dimensional transformation is:

$$\text{Input: } (H \times W \times C) \rightarrow \text{Patches: } (N \times P^2C) \rightarrow \text{Embeddings: } (N \times d_{\text{model}})$$

Positional encodings are added to the patch embeddings to preserve spatial order. A special learnable [CLS] token is prepended to the sequence; its representation at the output of the Transformer encoder is used as the aggregate image representation and passed to a classification head.

ViT offers several advantages over CNNs. Because self-attention operates globally from the first layer, ViT can capture long-range spatial dependencies between distant patches. The attention maps produced by ViT are directly interpretable as heat-maps over image patches, indicating which regions most influence each output decision. Furthermore, ViT scales extremely well on large datasets. The primary limitation of ViT is its data hunger: without pre-training on very large datasets, ViT typically underperforms comparable CNNs on mid-sized benchmarks, as CNNs benefit from useful inductive biases — local connectivity, weight sharing, and translation equivariance, that ViT must learn entirely from data.

6. Comparative Analysis: CNN vs. ViT on Brain MRI Classification

6.1 Problem Statement and Dataset

To empirically compare the behaviour of CNN and ViT models on a structured medical imaging task, both architectures were trained and evaluated on the Brain Tumor MRI Dataset (Kaggle, Massoud/Sartaj). The task is a four-class classification problem: given a brain MRI scan,

predict whether the tumour type is Glioma, Meningioma, Pituitary, or No Tumor. The dataset comprises 7,200 images in total, partitioned into 5,600 training images and 1,600 test images. Twenty percent of the training data was held out for validation. All images were resized to 224×224 pixels.

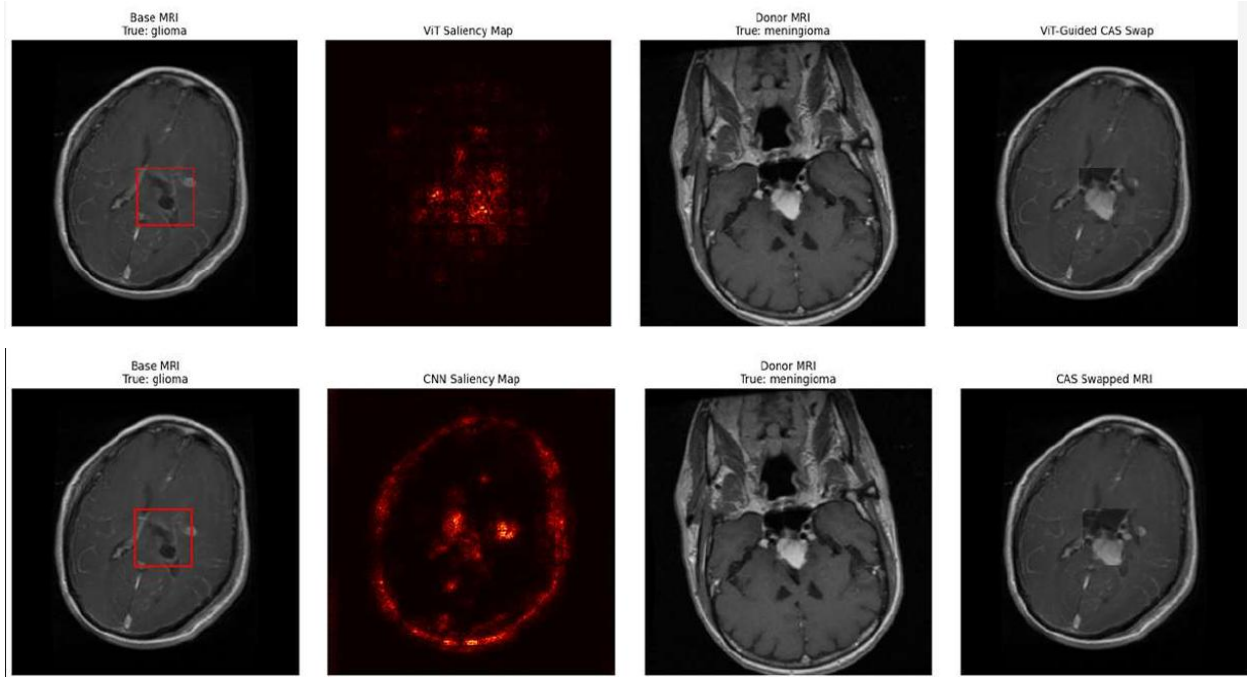


Figure 4: CNN is more sensitive to the modified region than ViT -indicating CNN relies more heavily on local spatial features..

6.2 Model Performance

The CNN model achieved a test accuracy of 85.88% and the ViT model achieved 92.75%. This gap of approximately seven percentage points is consistent with the general finding in the literature that, on tasks where sufficient data is available, the global attention mechanism of ViT provides representational advantages over the local feature extraction of CNNs. The higher accuracy of ViT is likely attributable to its ability to directly capture relationships between distant anatomical structures in a single attention step, rather than requiring deep stacks of convolutions to build up receptive fields large enough to see such patterns.

However, accuracy alone is insufficient to understand model behaviour, particularly in medical imaging contexts where safety, reliability, and interpretability are paramount. A model that achieves high accuracy by relying on spurious or non-anatomical features is fragile and

untrustworthy in deployment. This motivated the adoption of the Counterfactual Attention Swap (CAS) framework.

6.3 Counterfactual Attention Swap (CAS)

Counterfactual Attention Swap is an interpretability methodology that moves beyond saliency visualisation to test the causal influence of specific image regions on a model's predictions. The key question CAS asks is not simply which region a model attends to, but whether that region actually determines the prediction.

The CAS procedure operates as follows. First, a base image is selected and its most important region is identified using a saliency map — either from CNN gradient saliency or from ViT attention maps. Second, that region is physically replaced with the corresponding region from a donor image belonging to a different class. Third, both the CNN and the ViT are run on the modified image and the change in predicted class and confidence score (the confidence delta) are recorded. A large confidence delta indicates that the swapped region is causally important. Two variants are employed: CNN-guided CAS (region identified by CNN saliency) and ViT-guided CAS (region identified by ViT attention maps).

6.4 CNN-Guided CAS: Results

In a representative single-image CNN-guided CAS experiment, the base image was a Glioma scan and the salient region was identified by CNN gradient saliency. After the identified region was replaced with a patch from a Meningioma donor image, the CNN changed its prediction from Glioma to Meningioma with a confidence delta of +0.79, indicating high sensitivity to the swapped region. The ViT, by contrast, retained its Glioma prediction with a smaller confidence delta of +0.22, suggesting that ViT draws on more distributed evidence.

At the aggregate level, over 50 image pairs, the mean confidence delta for CNN was +0.1227 (± 0.2136) and for ViT was +0.1118 (± 0.3601). These values are not statistically significantly different. This aggregate result reveals that CNN-salient regions do not consistently matter more to the CNN than to the ViT: both models use distributed evidence, and the influence of any single region is highly case-dependent.

6.5 ViT-Guided CAS: Results

In the corresponding ViT-guided CAS single-image experiment, the same Glioma base image was used, but the salient region was now identified by the ViT attention map. After swapping this attention-guided region with a patch from a Meningioma donor, both the CNN and the ViT changed their predictions from Glioma to Meningioma, with confidence deltas of +0.7952 and +0.7448, respectively, a striking result suggesting that ViT attention focuses on regions that are anatomically and diagnostically meaningful rather than model-specific artefacts.

At the aggregate level over 50 images, the mean confidence delta for ViT in the ViT-guided setting was +0.0869 (± 0.2474), slightly higher than the CNN's mean delta of +0.0520 (± 0.1647) in the same setting. While the direction of this difference is consistent with expectation, it is not statistically significant at the aggregate level.

6.6 Discussion

The CAS results reveal several important insights. First, the expectation that CNN-guided regions would be more disruptive to CNN than to ViT, and vice versa, holds clearly in individual cases but does not persist as a statistically consistent aggregate pattern. Both models employ distributed representations, their decisions not determined by a single salient region. Second, ViT-guided regions appear more semantically meaningful; single-image ViT-guided CAS results show both models flipping their predictions, whereas CNN-guided CAS typically showed only the CNN flipping. Third, the overall superior accuracy of ViT (92.75% vs. 85.88%) combined with the CAS findings suggests that ViT not only performs better but reasons differently: its global attention integrates information from across the entire image simultaneously, which is advantageous when the diagnostic signal involves the spatial relationship between a tumour and remote anatomical structures. Finally, the consistently high variance in confidence deltas underscores that aggregate evaluation of interpretability methods is necessary to draw reliable conclusions about model behaviour.

7. Conclusion

This report has presented a unified treatment of two foundational deep learning architectures. The Convolutional Neural Network framework was developed from first principles, motivating the design of local weight-sharing filters as a solution to the spatial destruction caused by flattening, and tracing the architecture through convolutional layers, pooling layers, and the

fully connected classification head. The ResNet extension introduced shortcut connections that enabled training of networks of unprecedented depth by making the identity mapping the path of least resistance for gradient flow.

The Transformer architecture was introduced as a response to the fundamental computational and representational limitations of recurrent models. By replacing recurrence with self-attention, the Transformer achieved full parallelism, constant-length paths between any two sequence positions, and dramatically improved performance on sequence transduction tasks. The mechanism of scaled dot-product attention, multi-head attention, causal masking, and the encoder-decoder stack were developed in detail. A concrete English-to-Hindi translation experiment validated the theoretical claims about head specialisation and word-order learning.

The Vision Transformer demonstrated that the Transformer paradigm extends naturally to image classification by treating image patches as tokens, and achieves superior accuracy when sufficient training data is available, at the cost of the locality inductive biases that CNNs inherit for free.

The comparative study on brain MRI classification confirmed that ViT outperforms CNN in accuracy, and the Counterfactual Attention Swap analysis revealed that ViT attention regions are more causally influential and semantically aligned than CNN gradient saliency regions, while both models ultimately rely on distributed evidence. Future work should extend CAS analysis to larger datasets, investigate the degree to which multi-head specialisation in ViT correlates with anatomically meaningful regions across classes, and explore hybrid CNN-Transformer architectures.

8. References

- [1] Goodfellow I, Bengio Y, Courville A. Deep Learning. MIT Press; 2016.
- [2] Vaswani A, Shazeer N, Parmar N, Uszkoreit J, Jones L, Gomez AN, Kaiser L, Polosukhin I. Attention Is All You Need. Advances in Neural Information Processing Systems (NeurIPS). 2017; 30.
- [3] Dosovitskiy A, Beyer L, Kolesnikov A, Weissenborn D, Zhai X, Unterthiner T, Dehghani M, Minderer M, Heigold G, Gelly S, Uszkoreit J, Houlsby N. An Image is Worth 16×16 Words:

Transformers for Image Recognition at Scale. International Conference on Learning Representations (ICLR). 2021.

[4] Brain Tumor MRI Dataset. Kaggle. Available at:
<https://www.kaggle.com/datasets/masoudnickparvar/brain-tumor-mri-dataset>